RAG Chatbot

Technical Documentation

Author Name

March 31, 2026

This document describes the implementation, technologies, and operation of the RAG Chatbot application: a single-container FastAPI and React stack with Google Gemini and Vertex AI RAG Engine integration.

Contents

1 Introduction **3**

2 Technology Stack **3**
2.1 Backend (Python) . . . 3
2.2 Frontend (Node.js / React) . . . 3
2.3 Infrastructure and runtime . . . 3
2.4 AI and Google Cloud . . . 4

3 System Architecture **4**
3.1 High-level view . . . 4
3.2 Responsibilities . . . 4

4 Backend Implementation **5**
4.1 Project structure . . . 5
4.2 Configuration (`core/config.py`) . . . 5
4.3 Database layer (`core/database.py`) . . . 5
4.4 Data models (`models/chat.py`) . . . 5
4.5 API routes (`routes/chat.py`) . . . 6
4.6 RAG service (`services/rag_service.py`) . . . 6

5 Frontend Implementation **6**
5.1 Project structure . . . 6
5.2 Entry and app shell (`App.js`) . . . 7
5.3 Components . . . 7
5.4 API layer (`services/api.js`) . . . 7

6 Conversation Memory Strategy **7**

7 RAG Integration **8**

8 Deployment **8**
8.1 Dockerfile (multi-stage) . . . 8
8.2 Docker Compose . . . 8
8.3 Environment variables (reference) . . . 8

9 API Reference **8**

1 Introduction

The **RAG Chatbot** is a full-stack web application that provides a conversational interface similar to modern chat assistants. It combines:

- a **React** single-page user interface with sidebar, message history, and streaming replies;
- a **FastAPI** backend exposing REST APIs and **Server-Sent Events (SSE)** for token streaming;
- **Redis** for in-memory persistence of conversations and messages;
- **Google Gemini** (via the `google-genai` SDK and an API key) for text generation, summarization, and title generation;
- **Vertex AI RAG Engine** for retrieval-augmented generation: relevant chunks are fetched from a configured corpus and injected into the model context.

The application is **publicly accessible**: there is no login or JWT layer. A fixed logical user identifier (`default`) scopes all Redis data, which keeps the data model ready for future multi-user extensions without changing the API shape.

Key features:

- Conversation list, rename, delete, and new-chat flow.
- Streaming assistant responses over SSE without blocking other HTTP requests.
- Three-layer **conversation memory**: running summary of older turns, last 10 messages verbatim, and RAG context for the current query.
- Optional static serving of the built React app from the same origin in Docker (empty `REACT_APP_API_URL` at build time).

2 Technology Stack

2.1 Backend (Python)

Package	Version	Role
FastAPI	0.115.6	HTTP API, dependency injection, OpenAPI
Uvicorn	0.34.0	ASGI server (with standard extras)
redis[hiredis]	5.2.1	Async Redis client and C parser
Pydantic	2.10.4	Request/response validation
pydantic-settings	2.7.1	Environment-based configuration
google-genai	$\geq$1.0.0	Gemini client (generate, stream)
google-auth	$\geq$2.20.0	Service account OAuth2 for Vertex REST
httpx	$\geq$0.27.0	Async HTTP client for RAG REST calls

Table 1: Backend Python dependencies (from `requirements.txt`).

2.2 Frontend (Node.js / React)

2.3 Infrastructure and runtime

- **Docker**: multi-stage image — Node 22 Alpine builds the frontend; Python 3.13 Alpine runs the API, embedded Redis, and Uvicorn.
- **Docker Compose**: single service `chatbot`, port mapping, environment variables, optional read-only mount for GCP service account JSON.

Package	Version	Role
react / react-dom	18.3.x	UI framework
react-scripts	5.0.1	Create React App build toolchain
react-markdown	9.x	Markdown rendering in messages
remark-gfm	4.x	GitHub-flavored Markdown
react-syntax-highlighter	15.x	Code block highlighting (Prism)
react-icons	5.x	Icon set for sidebar and controls

Table 2: Frontend dependencies (from `package.json`).

- **Redis**: runs inside the container in production (`redis-server -daemonize yes`) with persistence disabled for simplicity (`-save "" -appendonly no`).

2.4 AI and Google Cloud

- **Gemini model** (configurable, default `gemini-2.5-flash`): chat completion, streaming, summarization, conversation titles.
- **Vertex AI RAG**: `retrieveContexts` REST API (`v1beta1`) against `RAG_CORPUS_NAME`, authenticated with a **service account** access token (not the Gemini API key).

3 System Architecture

3.1 High-level view

Figure 1 shows the main components and data flows.

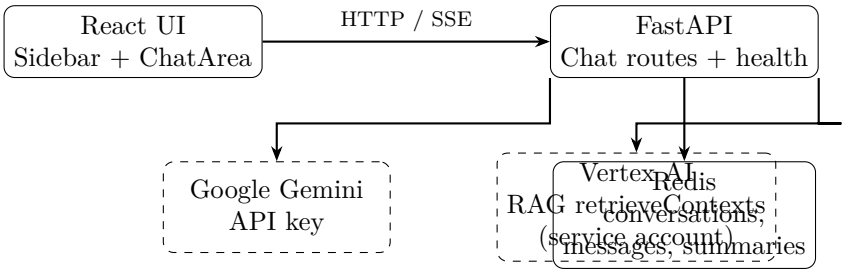

Figure 1: Logical architecture: browser, API, Redis, and external AI services.

3.2 Responsibilities

React frontend
Renders the chat layout, loads conversations and messages, sends user input to `POST /api/chat/send`, parses SSE lines for chunks and final `conversation_id`.

FastAPI backend
Validates requests, reads/writes Redis, orchestrates summarization and RAG retrieval, streams Gemini output as SSE events.

Redis
Stores conversation metadata (hash + sorted set index), message lists (JSON per element), per-conversation message ID counter, and optional running summary string.

Gemini
: Produces assistant text (streaming), incremental conversation summaries, and short titles for new threads.

Vertex RAG
: Returns ranked text chunks for the user query; the backend concatenates them into a single context block for the model.

4 Backend Implementation

4.1 Project structure

```
backend/
  requirements.txt
  app/
    main.py              # FastAPI app, CORS, lifespan, static SPA
    core/
      config.py          # Settings from environment
      database.py        # Redis pool init/close/get
    models/
      chat.py            # Pydantic schemas + Conversations/Messages tables
    routes/
      chat.py            # REST + SSE send endpoint
    services/
      rag_service.py     # Gemini, RAG REST, summarization, streaming
```

4.2 Configuration (`core/config.py`)

Settings are loaded with **pydantic-settings** from environment variables and optional `.env`. Relevant fields:

- `REDIS_URL` — Redis connection string.
- `GCP_PROJECT_ID`, `GCP_LOCATION`, `RAG_CORPUS_NAME` — Vertex RAG corpus resource name and routing.
- `GEMINI_MODEL_NAME` — Model id for generation.
- `GOOGLE_API_KEY` — Gemini API authentication.
- `GOOGLE_APPLICATION_CREDENTIALS` — Path to service account JSON for RAG REST calls.
- `CORS_ORIGINS` — Comma-separated list or `*`.
- `STATIC_DIR` — If set, enables mounting built React assets and SPA fallback routes.

`get_settings()` is cached with `lru_cache` for a single process lifetime.

4.3 Database layer (`core/database.py`)

`init_redis()` creates an async Redis client from `settings.REDIS_URL` with `decode_responses=True` and pings the server. `get_redis()` returns the global pool used by routes. `close_redis()` closes the connection on application shutdown (FastAPI lifespan).

4.4 Data models (`models/chat.py`)

Pydantic models define API contracts: `ConversationCreate`, `ConversationUpdate`, `ConversationResponse` `MessageResponse`, `ChatRequest`.

Redis key schema:

- `conv:{id}` — hash: `id`, `user_id`, `title`, `model`, `system_prompt`, `created_at`, `updated_at`.
- `user_convs:{user_id}` — sorted set: member conversation id, score = last update timestamp (for ordering).
- `msgs:{conv_id}` — list of JSON-serialized message objects (oldest first).
- `msg_counter:{conv_id}` — integer string for monotonic message ids.
- `conv_summary:{conv_id}` — string: running summary for context compression.

Table-class pattern: `ConversationsTable` and `MessagesTable` encapsulate Redis commands (mirroring a thin ORM layer), keeping route handlers declarative. Singleton instances `Conversations` and `Messages` are imported by routes.

4.5 API routes (`routes/chat.py`)

- `DEFAULT_USER = "default"` for all ownership checks.
- CRUD for conversations and listing messages under `/api/chat/...`.
- `POST /api/chat/send`: resolves or creates a conversation, appends the user message, optionally refreshes the summary, calls `retrieve_rag_context`, builds `context_messages`, streams SSE from `generate_streaming_response`, persists the assistant message, and may generate a title for brand-new threads. A final SSE event carries `conversation_id`.

Streaming uses an async generator that forwards chunks from the RAG service and collects the full assistant text in a `result_holder` list for persistence after the stream completes (avoiding re-parsing every SSE payload).

4.6 RAG service (`services/rag_service.py`)

- **`Initialization:`** Lazy singleton Gemini client from `GOOGLE_API_KEY`; optional service account credentials for Vertex.
- **`retrieve_rag_context(query):`** POST to `https://{location}-aiplatform.googleapis.com/v1beta1/projects/{project}/locations/{loca` with JSON body containing `vertex_rag_store` and `query` (text, top-k, optional distance filter). Parses `contexts` into a single string with source URIs when present.
- **`summarize_messages:`** Calls Gemini with a fixed summarizer prompt; supports folding in a previous summary.
- **`build_context_messages:`** Prepends system prompt, optional summary block, optional RAG block, then maps recent Redis messages to alternating user/model roles for the Gemini `contents` API.
- **`generate_streaming_response:`** Wraps synchronous streaming in `run_in_executor`, yields `data: {json}`
  `n`
  `n` SSE lines and appends full text to `result_holder`.
- **`generate_title_for_conversation:`** Short title from first user message; uses `(resp.text or "").strip()` to tolerate empty API responses.

5 Frontend Implementation

5.1 Project structure

```
frontend/
  public/index.html
  src/
    index.js, index.css
    App.js, App.css
```

```
    components/
      Sidebar.js, Sidebar.css
      ChatArea.js, ChatArea.css
    services/
      api.js
```

Empty directories `context/` and `pages/` may exist from earlier scaffolding and are unused.

5.2 Entry and app shell (`App.js`)

`App` holds state: conversation list, active conversation id, messages, streaming buffer, streaming flag, sidebar collapse. Effects load conversations on mount and reload messages when `activeId` changes. `sendMessageStream` receives chunks into a ref-backed buffer; on completion the assistant message is appended and the sidebar refreshes. Stopping generation clears streaming state and appends partial text with a stopped marker.

5.3 Components

Sidebar: Lists conversations (buttons for accessibility), search filter, new chat, collapse toggle, rename and delete actions calling REST helpers.

ChatArea: Empty state with suggested prompts; message list with `ReactMarkdown` and `remark-gfm`; code blocks via `react-syntax-highlighter` (Prism, oneDark); typing indicator while streaming with no tokens yet; auto-growing textarea; send vs stop button.

5.4 API layer (`services/api.js`)

`request()` wraps `fetch` with JSON and error handling. Exported functions: `getConversations`, `updateConversation`, `deleteConversation`, `getMessages`, `sendMessageStream`. The stream reader decodes lines starting with `data:` , parses JSON, dispatches `onChunk` for content deltas, collects errors and final `conversation_id`, then invokes `onError` and/or `onDone` after the stream ends.

`REACT_APP_API_URL` defaults to empty string for same-origin deployment; local dev sets `http://localhost:8000`.

6 Conversation Memory Strategy

For each `POST /api/chat/send`, the backend builds context as follows:

1. **Running summary** — Stored in Redis at `conv_summary:{id}`. When the total message count exceeds 10 and either no summary exists yet or the count is a multiple of 4, older messages (everything except the last 10) are sent to `summarize_messages` together with the prior summary; the result is written back to Redis.
2. **Recent window** — The last 10 messages (including the new user turn) are passed verbatim as chat history to Gemini.
3. **RAG context** — `retrieve_rag_context` uses the current user message text as the query; retrieved chunks are injected as a labeled block in `build_context_messages`.

This limits token growth on long threads while preserving salient facts in the summary and fresh detail in the last turns.

7 RAG Integration

- **Gemini:** Authenticated with `GOOGLE_API_KEY` through `google.genai.Client`.
- **Vertex RAG:** Requires `GOOGLE_APPLICATION_CREDENTIALS` pointing to a JSON key with permission to call Vertex AI in the project/region. The code exchanges credentials for an OAuth2 access token (with short-lived caching) and sends it in the `Authorization: Bearer` header.
- If the corpus or credentials are missing, RAG retrieval returns `None` and the model still answers using system prompt, summary, and recent messages only.
- Chunking and embedding are managed inside the RAG corpus (e.g. managed vector store in GCP); this application only queries the corpus by text.

8 Deployment

8.1 Dockerfile (multi-stage)

Stage 1 (`frontend-build`): `node:22-alpine`, `npm install`, copy sources, `ENV REACT_APP_API_URL=""`, `npm run build` → static files under `/build/build`.

Stage 2 (runtime): `python:3.13-alpine`, install `gcc`, `musl-dev`, `libffi-dev`, `redis`; `pip install -r requirements.txt`; copy backend; copy frontend build to `/app/static`; set `STATIC_DIR` and `REDIS_URL`; `start.sh` runs `redis-server` daemonized then `uvicorn app.main:app -host 0.0.0.0 -port 8000`.

8.2 Docker Compose

Service `chatbot`, build context project root, container name `rag-chatbot`, port `${PORT:-8000}:8000`, environment variables for Redis, GCP, Gemini, CORS, static dir, volume `${GCP_SA_KEY_PATH:-/dev/null}:/app/credentials/sa-key.json:ro`.

8.3 Environment variables (reference)

Variable	Description
`REDIS_URL`	Redis URL (container: `redis://localhost:6379/0`).
`GCP_PROJECT_ID`	Google Cloud project id.
`GCP_LOCATION`	Vertex region (e.g. `europe-west3`).
`RAG_CORPUS_NAME`	Full resource name of the RAG corpus.
`GEMINI_MODEL_NAME`	Gemini model id.
`GOOGLE_API_KEY`	API key for Gemini (Generative Language API).
`GOOGLE_APPLICATION_CREDENTIALS`	Filesystem path to service account JSON inside the container.
`CORS_ORIGINS`	Allowed origins (comma-separated) or `*`.
`STATIC_DIR`	Path to built SPA (`/app/static` in Docker).
`PORT`	Host port for Compose (optional, default 8000).
`GCP_SA_KEY_PATH`	Host path to key file mounted at `/app/credentials/sa-key.json`.

9 API Reference

Method	Path	Description
GET	`/api/health`	Health check; returns `{"status": "ok"}`.
GET	`/api/chat/conversations`	List all conversations (default user).
POST	`/api/chat/conversations`	Create a conversation (optional; UI creates via send).
GET	`/api/chat/conversations/{id}`	Get one conversation metadata.
PUT	`/api/chat/conversations/{id}`	Update title, model, or system prompt fields.
DELETE	`/api/chat/conversations/{id}`	Delete conversation and related Redis keys.
GET	`/api/chat/conversations/{id}/messages`	List messages in order.
POST	`/api/chat/send`	Body: `{ "message": string, "conversation_id": string|null }`. Response: `text/event-stream` SSE (content chunks, then `conversation_id`).

SSE payload shapes (JSON after `data:`):

- Streaming chunk: `{"content": "<delta>", "done": false}`
- Stream finished: `{"content": "", "done": true}`
- Error: `{"error": "<message>"}`
- Final: `{"conversation_id": "<uuid>"}`

Line `data: [DONE]` may appear after completion.